

AI Buyer Watchdog — Hackathon PRD

Sep 26, 2026 · @Darshan

TL;DR

We are building an AI mystery shopper for online stores: enter a store URL, our agent tests whether AI buyers can **see**, **find**, and **buy** from it, and a dashboard shows the exact step where the sale dies. One line: *uptime monitoring, but for AI buyers*.

- **Working name:** AI Buyer Watchdog (rename freely)
- **Event:** Grok Bot Commerce London Hackathon, Fleek — Saturday 26 Sep 2026
- **Track:** Merchant Tooling (overlaps Agentic Commerce)
- **Hard deadlines:** code freeze + submission **16:30** via the site's Submit button; Top-5 live demos (3 min) at 17:30
- **Non-negotiable demo moment:** the agent fails to buy on camera, we fix the store, the agent succeeds

Problem

Merchants are silently losing sales to AI buyers, and nothing in their analytics tells them. When ChatGPT, Grok, or Perplexity shops on a user's behalf and can't read or buy from a store, it simply buys from a competitor — no error, no bounce, no trace.

- AI assistants are becoming a real commerce channel (ChatGPT Instant Checkout, Grok agents, Perplexity shopping), so the losses grow every month.
- Most modern storefronts render product data with JavaScript, but most AI fetchers **don't execute JavaScript** — to them the store is a blank page.
- Checkout flows are full of agent-killers: newsletter popups, forced sign-in, CAPTCHAs, fragile forms.
- Existing tools (Shopify's readiness checker, Visibl and similar) are **static checklists**. None of them shows the merchant the actual behavioral outcome: "an agent tried to buy from you today and here is where it died."

The merchant's question we answer: "**Can an AI buy from my store right now — and if not, why?**"

How AI buyers shop today (and what we mirror)

AI assistants touch a store in four layers, and each layer fails differently. Our product tests the store the same way agents actually behave, so the audit reflects reality — not a made-up checklist.

Layer	What happens	Typical failure
1. Background crawl	GPTBot / ClaudeBot / Grok's crawler read the site to learn the brand	Blocked by robots.txt, thin content
2. Live fetch at question time	Assistant fetches pages as raw HTML — mostly without running JavaScript	JS-only storefront = blank page to the AI
3. Shopping systems	Product cards picked from schema.org structured data + merchant product feeds (OpenAI feed spec, ACP)	Missing/incomplete schema → never listed
4. Agent browser mode	At purchase time an agent drives a real browser: click, fill, check out	Popups, forced login, CAPTCHAs, broken forms

Key evidence: [Vercel's AI crawler study](#) and [2026 follow-ups](#) confirm the JS gap; [OpenAI's shopping doc](#) confirms structured data + feeds drive product selection.

Solution

Store URL in → three checks that mirror the four layers → one dashboard verdict: **Seen / Listed / Buyable**, plus the exact failing step and a plain-English fix list.

Check	Question it answers	How it works
Seen	What does an AI fetcher actually get from your pages?	Fetch product page with JavaScript off; diff visible product info vs what's in raw HTML. Safe to run on any real store, live
Listed	Can shopping systems list your products?	Detect schema.org Product markup / product feed presence and completeness

Check	Question it answers	How it works
Buyable	Can an agent complete a purchase?	Playwright + Grok-driven agent runs the journey: home → search → product → add to cart → checkout, logging + screenshot at every step

Money safety (hard rules): on any live store the agent stops **before** "Place Order" — roughly 95% of failures happen earlier anyway. Full end-to-end purchase only on our demo store using Stripe/Shopify **test mode** (test card 4242...). No real money ever moves.

Output: one dashboard page — three verdicts, a step timeline with screenshots, the failing step highlighted, and the fix list (e.g. "render product data server-side", "add Product schema", "remove popup on checkout").

MVP scope

We build exactly four things; everything else is roadmap. The only non-negotiable is the agent failing, then succeeding, on camera.

1. **Demo store with a broken ↔ fixed toggle** — small Next.js (or Shopify dev) store, test-mode checkout. Broken mode: JS-only rendering, no Product schema, popup + forced sign-in at checkout. Fixed mode: SSR, schema present, clean guest checkout.
2. **Agent runner** — Playwright driven by Grok (any LLM as fallback): search → product → cart → checkout; logs a verdict + screenshot per step; stops at payment except in test mode.
3. **Seen check** — JS-off fetch + diff; works live on any real store URL.
4. **Dashboard (one page)** — Seen / Listed / Buyable verdicts, step timeline with screenshots, failing step highlighted, fix list.

Cut (say "roadmap" with confidence): auto-fix AI (the toggle simulates its effect), scheduled/daily runs and alerts, real-store checkout completion, feed generation, auth/multi-store, passive detection of real agent traffic in logs.

Stack: Next.js + Playwright + Grok API; build in Cursor (sponsor credits); deploy on Vercel only if trivial — local is fine. Supabase optional; skip if it costs more than 15 minutes.

Team split (2 people): one owns demo store + dashboard; one owns agent runner + Seen check. Integrate over a single shared JSON result format agreed up front.

3-minute demo script

One story, four beats: lose the sale, see why, fix it, win it back.

1. **Hook (30s):** "AI agents are shopping right now. Can they buy from you?" Run the Seen check live on a well-known real store → show the blank/thin page an AI fetcher actually gets.
2. **The lost sale (60s):** the agent shops our demo store in broken mode — audience watches it die at the checkout popup. Dashboard lights up: Buyable , failing step highlighted with screenshot.
3. **Fix and win (60s):** flip the fixed toggle ("this is what our fix list produces") → re-run → agent completes the order in test mode → Buyable .
4. **Close (30s):** "Merchants have uptime monitoring for their site. Nothing watches the AI buyer that visits at 3am. We're that watchdog — roadmap: auto-fix, daily runs, alerts."

Backup: a pre-recorded GIF of a full successful run, in case the live agent flakes on stage.

Judging criteria mapping

Every team submits a live demo at code freeze; the Top-5 highest-scoring teams present live and judges award 1st–3rd. How we score on all seven criteria:

Criterion	Our answer
Problem	Merchants silently lose sales to AI buyers — invisible in analytics, growing monthly
Experience	Merchant-side impact: a fix list that directly recovers lost revenue
Execution	Deliberately tiny surface (4 components) so the live run actually works
AI / Grok Bot	The agent IS the product; Grok is the shopper's brain — on-theme by construction
Commerce depth	Touches storefront rendering, schema.org/product feeds, and checkout mechanics
Originality	Everyone builds chatbots; we build monitoring. Shopify's static checker proves the market — we're behavioral (live agent, real outcome)
Impact	"Protect the fastest-growing sales channel" — plausible business value the VC judges will get

Submission: via the hackathon site's Submit button **before 16:30**. Include: one-line pitch, demo store URL, dashboard URL/screenshot, repo link, backup GIF.

Timeline (plan written at 13:50 — 2h40m to freeze)

Build in this order; the agent run is the only thing that cannot be cut.

Time	Task
13:50–14:20	Demo store with broken/fixed toggle, test-mode checkout
14:20–15:20	Agent runner: Playwright + Grok loop, verdict + screenshot per step
15:20–15:45	Seen check (JS-off fetch + diff), results into shared JSON
15:45–16:10	Dashboard page: verdicts, timeline, failing step
16:10–16:30	Record backup GIF, polish, Submit

Cut-line if slipping (in order): drop the Listed/schema check → simplify dashboard to a plain results list → hardcode the demo-store product. Never cut the fail→fix→succeed agent run.

Risks and guards:

- Live agent flakiness → backup GIF recorded at 16:10; demo runs against the local store, zero venue-wifi dependency.
- Time overrun on the store → use a template/starter, hardcode one product; the store is a prop, not the product.
- "Is the fix real?" question from judges → honest answer: the toggle shows the effect of the fix list; automated fixing is roadmap.

Market evidence & roadmap

The pain is proven by real products; the behavioral angle is the open gap we take.

- Static readiness checkers exist — [Shopify's own free tool](#) (31 checks), [Visibl](#) (explicitly "a diagnostic scanner, not a behavioral simulator"), [Agentic Commerce Readiness](#), [AgentIQ](#), [Tacey](#) — market validated, none behavioral.
- "Tell merchants what to do" analytics is a funded category — [Triple Whale](#) and its [agentic pivot](#).
- The JS gap is documented — [Vercel](#), [2026 tests](#).
- Agent traffic is called the [new analytics blind spot](#); shopping selection runs on [feeds + schema](#).

Roadmap (post-hackathon): auto-fix agent that applies the fix list; scheduled daily runs with Slack/email alerts; run against any real store URL; competitor benchmarking ("agent picked them because..."); ACP/feed readiness checks; passive detection of real agent visits in traffic logs.